Contents

1 The GPU Efficiency Gap: A Systematic Method for Detecting Invisible Waste 2
1.1 Abstract . . . 2
1.2 1. Introduction . . . 3
1.2.1 1.1 Key Findings . . . 3
1.2.2 1.2 Contributions . . . 4
1.2.3 1.3 Scope . . . 4
1.3 2. Background and Related Work . . . 4
1.3.1 2.1 GPU Virtualization Technologies . . . 4
1.3.2 2.2 The Monitoring Gap . . . 4
1.3.3 2.3 Performance Measurement and the Roofline Model . . . 5
1.3.4 2.4 Distributed Training and the Straggler Problem . . . 5
1.4 3. Waste Taxonomy and Measurement Model . . . 5
1.4.1 3.1 Ghost Allocations . . . 5
1.4.2 3.2 Contention Squeeze . . . 6
1.4.3 3.3 Provisioning Overhead . . . 6
1.4.4 3.4 Burst-to-Sustained Gap . . . 6
1.4.5 3.5 Straggler Tax . . . 6
1.4.6 3.6 Oversubscription Blind Spots . . . 6
1.4.7 3.7 Noise Model . . . 7
1.5 4. Simulation Infrastructure and Experimental Design . . . 7
1.5.1 4.1 Methodology . . . 7
1.5.2 4.2 Hardware-Validated Foundation . . . 7
1.5.3 4.3 Experimental Design Summary . . . 8
1.5.4 4.4 Detection Model . . . 8
1.6 5. Results . . . 8
1.6.1 5.1 Overview . . . 8
1.6.2 5.2 All Six Waste Categories Are Detectable . . . 8
1.6.3 5.3 Current Tools Provide Zero Visibility . . . 9
1.6.4 5.4 Operational Impact: Three Action Buckets . . . 11
1.6.5 5.5 Hypothesis Summary . . . 12
1.7 6. Discussion . . . 13
1.7.1 6.1 A Monitoring Paradigm Shift . . . 13
1.7.2 6.2 Operational Priorities . . . 14
1.7.3 6.3 Limitations and Validity . . . 14
1.8 7. Reproducibility . . . 15
1.8.1 7.1 Artifact Inventory . . . 15
1.8.2 7.2 Rerun Commands . . . 15
1.9 8. Conclusion . . . 15
1.10 References . . . 16

2 The Hidden Cost of GPU Virtualization 16
2.1 Simulation Dataset . . . 16
2.2 Table 1. What Visibility Enables — Three Action Buckets . . . 16
2.3 Table 2. Detection Rates by Tool Across All 6 Waste Categories . . . 17
2.4 Table 3. Cross-Category Omnibus Statistical Tests . . . 18
2.5 Table 4. Scenario-Based Annual Impact by Category and Fleet Scale . . . 18

2.6 Cost Model Bootstrap Confidence Intervals . . . 19
2.7 Interpretation Guardrails . . . 19

3 Supplementary Tables **19**
3.1 Table S1. Descriptive Statistics by Category and Condition . . . 19
3.2 Table S2. Full Statistical Test Results . . . 26
3.3 Table S4. Cost Model Parameter Sensitivity . . . 29
3.4 Table S5. Tool Detection Confusion Matrices . . . 29
3.5 Straggler Threshold Sweep . . . 31

1 The GPU Efficiency Gap: A Systematic Method for Detecting Invisible Waste

Christopher D. Jones

March 2026

1.1 Abstract

Background. GPU fleets cost millions of dollars annually. The standard monitoring tools, `nvidia-smi` and NVIDIA DCGM, cannot observe the efficiency losses that accumulate across device thermal physics, virtualization partitioning, and fleet-level coordination. These tools were designed for single-device health checks. They do not measure lifecycle transitions, cross-tenant interference, thermal trajectories, or distributed synchronization waste. The result is a systematic observability gap: operators manage GPU infrastructure without visibility into losses that span the full operations stack.

Methods. We formalize six categories of invisible GPU waste covering single-device thermal physics, virtualization partitioning, and fleet-level coordination. We employ a synthetic model calibrated against hardware-validated roofline measurements on H100 SXM (2,905 GB/s HBM3, 59.1 TFLOPS FP32, validated at 87 to 100% of spec) and H200 systems. The simulation executes 120,000 deterministic trials (20,000 per category) with NVML-matched noise injection. We apply Mann-Whitney U for independent-group categories and Wilcoxon signed-rank for paired within-trial categories, with Holm-Bonferroni correction across all six omnibus comparisons.

Results. All six waste categories produce statistically significant effects with large standardized effect sizes (Cohen's d/d_z ranging from 0.73 to 8.55). `nvidia-smi` and DCGM detect 0% of waste events across all categories. The `gpu-roofline` framework detects 56.5 to 100% (McNemar $p < 1e\text{-}300$ for all comparisons). The waste categories map to three operational responses: (A) directly recoverable capacity, including ghost allocations freeing 512 MiB per teardown and straggler detection recovering 19% of fleet throughput; (B) decision support for infrastructure design, including contention data for MIG versus time-slicing decisions and thermal data for accurate SLAs; and (C) risk prevention through oversubscription detection before silent tenant degradation.

Conclusions. We provide strong experimental evidence that six categories of measurable GPU efficiency loss are completely invisible to the standard monitoring tools deployed across the industry. This study establishes the taxonomy, the detection method, and a reproducible benchmark built on

hardware-validated performance models. The open protocol and predictions are published to enable community hardware validation.

Keywords: GPU efficiency, invisible waste, roofline model, observability gap, MIG, virtualization, fleet operations, distributed training

---

1.2 1. Introduction

GPU infrastructure represents one of the largest capital expenditures in modern computing. A single NVIDIA H100 GPU costs approximately $2.50 per hour in the cloud, or $22,000 per year. At fleet scale, these costs reach into the hundreds of millions. The tools used to monitor this infrastructure, `nvidia-smi` and NVIDIA DCGM, were designed for a fundamentally different purpose: single-device health monitoring. They report whether a GPU is functional and whether it is overheating. They do not report where efficiency is being lost.

This distinction matters because GPU efficiency loss spans the full operations stack. At the device level, thermal throttling silently reduces sustained performance below advertised specifications. At the virtualization level, memory remains trapped after partition teardown and bandwidth degrades under tenant contention. At the fleet level, a single degraded GPU forces an entire distributed training cluster to idle at synchronization barriers. These losses are not small. They are invisible because the monitoring architecture lacks the measurement primitives to observe them.

Three scenarios illustrate the scope of this problem.

First, consider device-level thermal degradation. A cloud provider advertises H100 instances at peak specifications. Under sustained workloads, thermal throttling reduces actual performance by 1 to 16% depending on workload type. Neither the provider nor the tenant can observe this gap because no tool tracks the thermal trajectory from burst clock to thermal equilibrium.

Second, consider virtualization lifecycle waste. A MIG partition is destroyed, but 512 MiB of VRAM is not reclaimed by the driver. The memory is physically consumed but allocated to no instance. `nvidia-smi` reports the partition as cleanly removed. Over 20 teardowns per day, gigabytes of sellable capacity silently disappear.

Third, consider fleet-level synchronization loss. A distributed training job across 8 H100 GPUs takes 40% longer than expected. One GPU has degraded NVLink bandwidth. Data-parallel training synchronizes at gradient barriers, so the other 7 GPUs sit idle waiting for the straggler. Each GPU individually reports normal utilization. No per-device tool flags the fleet-wide impact.

These are not edge cases. They are structural consequences of how GPU infrastructure operates. Existing monitoring tools report instantaneous per-device state rather than tracking transitions across time, across tenants, and across devices.

This paper presents a systematic method for identifying these efficiency losses. We formalize six categories of invisible GPU waste spanning the full operations stack and demonstrate that all six are detectable with purpose-built instrumentation where existing tools see nothing.

1.2.1 1.1 Key Findings

Finding 1: The observability gap is total. Across 120,000 trials spanning six waste categories, modeled `nvidia-smi` and DCGM detection rates are 0.0%. The `gpu-roofline` measurement

framework detects 56.5 to 100% of waste events depending on category. Current tools lack the architectural capability to observe GPU efficiency loss beyond instantaneous device state.

Finding 2: The waste is real and measurable. All six categories produce statistically significant effects with large standardized effect sizes after Holm-Bonferroni correction (d/d_z = 0.73 to 8.55). These represent 512 MiB of trapped VRAM per teardown, 19% fleet throughput loss per straggler GPU, and 50 to 75% bandwidth degradation per tenant under time-slicing.

Finding 3: The six categories require three different operational responses. Ghost allocations and straggler GPUs represent directly recoverable capacity. Contention and thermal throttling are governed by physics. They cannot be eliminated, but measuring them enables better infrastructure decisions including MIG versus time-slicing selection and accurate SLA pricing. Oversubscription is a preventable risk that monitoring can intercept before tenants are impacted. The value of visibility is category-specific.

1.2.2 1.2 Contributions

1. A six-category taxonomy of invisible waste across the GPU operations stack (device, virtualization, fleet), with formal mechanisms and detection requirements for each.
2. A reproducible simulation harness producing 120,000 deterministic trials with byte-identical rerun capability (SHA-256 verified).
3. Design-appropriate statistical analyses employing Mann-Whitney U for independent groups, Wilcoxon signed-rank for paired measurements, Holm-Bonferroni correction, and bootstrap confidence intervals.
4. A three-bucket operational impact model that distinguishes recoverable capacity from decision support from risk prevention.
5. An open protocol and simulation benchmark for community hardware validation.

1.2.3 1.3 Scope

This study establishes detectability and relative magnitude through simulation. Absolute real-world magnitudes require hardware validation, which we reserve for a follow-on study and enable through the published open protocol.

1.3 2. Background and Related Work

1.3.1 2.1 GPU Virtualization Technologies

NVIDIA Multi-Instance GPU (MIG), introduced with the A100 architecture, provides hardware-isolated partitions with dedicated compute, memory, and cache resources. Each MIG instance operates as an independent GPU with guaranteed quality of service. NVIDIA GRID/vGPU enables time-sliced sharing where tenants alternate access to the full GPU, achieving higher tenant density at the cost of bandwidth contention. Both technologies are widely deployed in cloud infrastructure from AWS, GCP, Azure, and Oracle Cloud [1].

1.3.2 2.2 The Monitoring Gap

The standard GPU monitoring stack consists of `nvidia-smi` (the NVML command-line interface) and NVIDIA Data Center GPU Manager (DCGM). Both provide per-device and per-instance metrics including utilization, memory consumption, temperature, power, and ECC error counts. These tools were designed for single-device health monitoring.

Multi-tenant waste requires a fundamentally different class of measurement:

Waste Type	Required Measurement	nvidia-smi/DCGM Capability
Lifecycle transitions	Pre/post memory delta across teardown	Reports only post-state
Cross-tenant interference	Per-tenant bandwidth before/after new arrival	Reports only aggregate per-GPU
Temporal degradation	Burst vs. sustained performance trajectory	Reports only instantaneous snapshots
Fleet aggregation	Sum of all vGPU allocations vs. physical capacity	Reports only per-instance
Distributed synchronization	Fleet-wide barrier wait analysis	Reports only per-device

The table above identifies an architectural gap that leaves an entire class of operational waste unmeasured.

1.3.3 2.3 Performance Measurement and the Roofline Model

The roofline model [2] provides a well-established framework for analyzing compute and memory bandwidth bounds. The `gpu-roofline` tool extends this framework with a dynamic roofline that captures both burst performance (peak clock, cold GPU) and sustained performance (thermally throttled equilibrium). The gap between these two ceilings represents capacity that is advertised but not sustainably deliverable. We term this the burst-to-sustained gap.

1.3.4 2.4 Distributed Training and the Straggler Problem

Data-parallel distributed training requires all-reduce synchronization at gradient aggregation barriers. The slowest GPU determines effective throughput for the entire fleet [3]. Prior work has focused on software mitigations including redundant computation and asynchronous SGD. Our contribution addresses the detection side: identifying which GPU is the straggler, diagnosing the cause of degradation, and quantifying fleet-wide impact.

1.4 3. Waste Taxonomy and Measurement Model

We define six categories of invisible waste across the GPU operations stack. Each category is characterized by a physical mechanism, a reason existing tools miss it, and a measurement capability required for detection.

1.4.1 3.1 Ghost Allocations

After vGPU teardown, VRAM may not be fully reclaimed by the driver. The unreleased memory sits physically consumed but unallocated to any instance. `nvidia-smi` reports the partition as removed without verifying that physical memory returned to baseline. Detecting this ghost requires pre-teardown memory capture via `nvmlDeviceGetMemoryInfo().used`, post-teardown polling with a stabilization protocol where readings converge within 1 MiB, and delta computation.

We simulate 10,000 treatment trials with ghost bytes injected as Uniform(0, 1024 MiB) across 3 teardown methods and 3 MIG profiles, against 10,000 zero-injection controls.

1.4.2 3.2 Contention Squeeze

When a new tenant arrives on a time-sliced GPU, every existing tenant loses bandwidth. Each receives approximately 1/N of total memory bandwidth, where N is the tenant count. `nvidia-smi` continues reporting 100% GPU utilization because it has no concept of per-tenant bandwidth. The aggregate number masks the per-tenant degradation entirely.

Detection requires per-tenant baseline capture before each new tenant arrives, post-arrival measurement, and comparison. We simulate 10,000 treatment trials across 3 tenant counts under time-slicing, against 10,000 control trials at single-tenant baseline plus MIG-partitioned configurations.

1.4.3 3.3 Provisioning Overhead

MIG partition creation takes 120 to 500 ms of wall-clock time depending on profile size and GPU load state. During this interval, capacity is neither free nor usable. `nvidia-smi` reports state only after the command returns, missing the transition entirely.

Detection requires wall-clock timing from create command to first successful compute dispatch. We simulate 20,000 paired trials measuring both true spin-up latency and nvidia-smi reported latency (approximately 0.5 ms).

1.4.4 3.4 Burst-to-Sustained Gap

GPUs boost to peak clock at cold start and thermally throttle to a lower sustained clock within 30 to 120 seconds. The gap between burst and sustained performance ranges from 1 to 16% for datacenter GPUs (H100, H200) and is larger for consumer hardware. Monitoring tools take instantaneous snapshots. They never track the thermal trajectory from burst to equilibrium, so the gap between advertised and deliverable performance remains invisible.

We simulate 20,000 paired trials measuring gap percentage versus ideal 0% across 5 GPU profiles and 3 workload types. The thermal model follows Newton's law of cooling with per-GPU coefficients derived from hardware measurements.

1.4.5 3.5 Straggler Tax

One degraded GPU forces N-1 healthy GPUs to idle at every synchronization barrier in data-parallel training. Five hardware degradation types contribute: thermal paste failure, NVLink degradation, PCIe fallback, memory subsystem failure, and clock stuck. Each GPU individually reports normal or near-normal metrics. No tool correlates fleet-wide performance to identify the barrier bottleneck.

Detection requires fleet-wide measurement, median-versus-outlier comparison, and per-GPU diagnostic probing. We simulate 10,000 treatment trials with degradation injected into one GPU across 5 types, 3 severities, and 3 fleet sizes (8, 32, 128), against 10,000 control trials with no degradation.

1.4.6 3.6 Oversubscription Blind Spots

Hypervisors can allocate more aggregate vGPU VRAM than physical GPU capacity. At overcommit ratios above 1.0, tenants experience performance degradation proportional to the excess. Each allocation appears successful individually. No tool sums allocations against physical limits, so the overcommit remains invisible until tenants experience degradation or out-of-memory crashes.

We simulate 10,000 treatment trials at overcommit ratios of 1.25x, 1.5x, and 2.0x, against 10,000 control trials at 1.0x.

1.4.7 3.7 Noise Model

All simulations include realistic measurement noise calibrated to NVML instrumentation characteristics:

Noise Source	Distribution	Parameters
NVML memory readings	Gaussian	sigma = 512 KiB
Spin-up latency	Log-normal	sigma = 0.3
Bandwidth measurement	Gaussian	sigma = 2% of baseline
Background memory spikes	Poisson x Uniform	lambda = 0.01, 1 to 64 MiB
Thermal readings	Gaussian	sigma = 1.0 C
Fleet performance	Gaussian	sigma = 2% of peak

1.5 4. Simulation Infrastructure and Experimental Design

1.5.1 4.1 Methodology

We employ a synthetic model to systematically evaluate all six waste categories under controlled conditions. We implement the simulation framework in Rust as part of the `gpu-harness` crate, comprising approximately 2,600 lines across 6 modules. The binary accepts three parameters: scale factor (default 1.0, producing 120,000 trials), random seed (default 42), and output path. Execution time is 0.4 seconds on commodity hardware. The output is deterministic: re-execution with the same seed produces byte-identical JSON (SHA-256: `e386a3d599a238901abff636d421dbcb095cca31616e425d05e5653744fd3912`).

1.5.2 4.2 Hardware-Validated Foundation

We calibrate the simulation models against measured hardware performance, not theoretical specifications. The dynamic roofline model underpinning the burst-to-sustained gap analysis (Category 4) and the straggler detection framework (Category 5) has been validated on bare-metal systems:

GPU	Measured HBM BW	Measured FP32	% of Theoretical	Platform
H100 SXM 80GB	2,905 GB/s	59.1 TFLOPS	87 to 100%	RunPod bare-metal
H200 SXM 141GB	4,028 GB/s	59.5 TFLOPS	87 to 100%	RunPod bare-metal
RTX 5090 32GB	1,503 GB/s	95.8 TFLOPS	87 to 100%	Cloud instance

We derive the thermal model (Newton's law of cooling with per-GPU thermal coefficients) and power model (workload intensity to clock frequency mapping) from these validated profiles. GPU profiles

used in the simulation, including clock speeds, TDP, thermal throttle onset temperatures, and bandwidth specifications, match hardware-measured values rather than vendor datasheet maximums. Full validation reports with measurement methodology, coefficient of variation, and NVML telemetry are archived in the repository (`docs/validation/`).

This hardware calibration is critical. It establishes that the simulation's performance predictions for burst clocks, sustained throughput, and thermal trajectories are grounded in measured physics.

1.5.3 4.3 Experimental Design Summary

Category	N	Design	Primary Test
1. Ghost allocations	20,000	Independent groups	Mann-Whitney U
2. Contention squeeze	20,000	Independent groups	Mann-Whitney U
3. Provisioning overhead	20,000	Paired (within-trial)	Wilcoxon signed-rank
4. Burst-sustained gap	20,000	Paired (within-trial)	Wilcoxon signed-rank
5. Straggler tax	20,000	Independent groups	Mann-Whitney U
6. Oversubscription	20,000	Independent groups	Mann-Whitney U

The choice of statistical test follows from the experimental design. Categories 3 and 4 produce paired observations where each trial measures both the actual value and the tool-reported or ideal baseline, requiring a paired test. We apply Holm-Bonferroni correction across all six omnibus p-values (family-wise alpha = 0.05).

1.5.4 4.4 Detection Model

For each trial, we record three binary detection outcomes: whether `gpu-roofline`, `nvidia-smi`, or DCGM would detect the waste event. We model `nvidia-smi` and DCGM detection at 0% based on architectural capability analysis. These tools lack the measurement primitives described in Section 3. Live tool comparison is deferred to hardware validation.

1.6 5. Results

1.6.1 5.1 Overview

The results support all three study hypotheses. All six waste categories produce large, statistically significant effects. Existing monitoring tools detect none of them. The six categories map to three distinct operational responses. We present the results in three sections: omnibus statistical outcomes (5.2), the observability gap (5.3), and the three-bucket operational model (5.4).

1.6.2 5.2 All Six Waste Categories Are Detectable

All six categories of invisible waste produce measurable, statistically significant effects. Table 1 presents the omnibus results.

Table 1. Omnibus Statistical Results Across All Six Waste Categories

Category	Design	Test	d / d_z	r	95% CI of Median Difference
Ghost allocations	Independent	Mann-Whitney U	2.46	1.00	[527M, 545M] bytes
Contention squeeze	Independent	Mann-Whitney U	8.55	1.00	[66.61, 66.68]% bandwidth loss
Provisioning overhead	Paired	Wilcoxon signed-rank	2.01	1.00	[244, 248] ms hidden latency
Burst-sustained gap	Paired	Wilcoxon signed-rank	0.73	1.00	[1.63, 1.77]% below spec
Straggler tax	Independent	Mann-Whitney U	2.46	1.00	[14.56, 15.09]% fleet throughput
Oversubscription	Independent	Mann-Whitney U	3.97	1.00	[33.33, 33.33]% degradation

All Holm-Bonferroni adjusted p < 1e-300. N = 10,000 to 20,000 per group. Effect sizes (d, d_z) and bootstrap 95% CIs (10,000 resamples) are the primary measures. P-values are reported for completeness but are uninformative at this sample size.

Effect sizes are uniformly large (Figure 2). The weakest category, burst-to-sustained gap at d_z = 0.73, still qualifies as a medium-to-large effect by conventional benchmarks. The strongest, contention squeeze at d = 8.55, reflects the deterministic nature of bandwidth partitioning under time-slicing.

1.6.3 5.3 Current Tools Provide Zero Visibility

We predicted that existing tools would miss greater than 80% of waste events. The result is more extreme. Modeled detection is 0% across all six categories for both `nvidia-smi` and DCGM.

Table 2. Detection Rates by Monitoring Tool

Category	nvidia-smi	DCGM	gpu-roofline	McNemar p
Ghost allocations	0.0%	0.0%	99.9%	<1e-300
Contention squeeze	0.0%	0.0%	100.0%	<1e-300
Provisioning overhead	0.0%	0.0%	100.0%	<1e-300
Burst-sustained gap	0.0%	0.0%	56.5%	<1e-300
Straggler tax	0.0%	0.0%	94.7%	<1e-300
Oversubscription	0.0%	0.0%	100.0%	<1e-300
Overall	**0.0%**	**0.0%**	**88.4%**	

The 0% detection rate for `nvidia-smi` and DCGM is a modeling result based on documented tool capabilities (see Section 4.4). These tools do not compute memory deltas across teardowns,

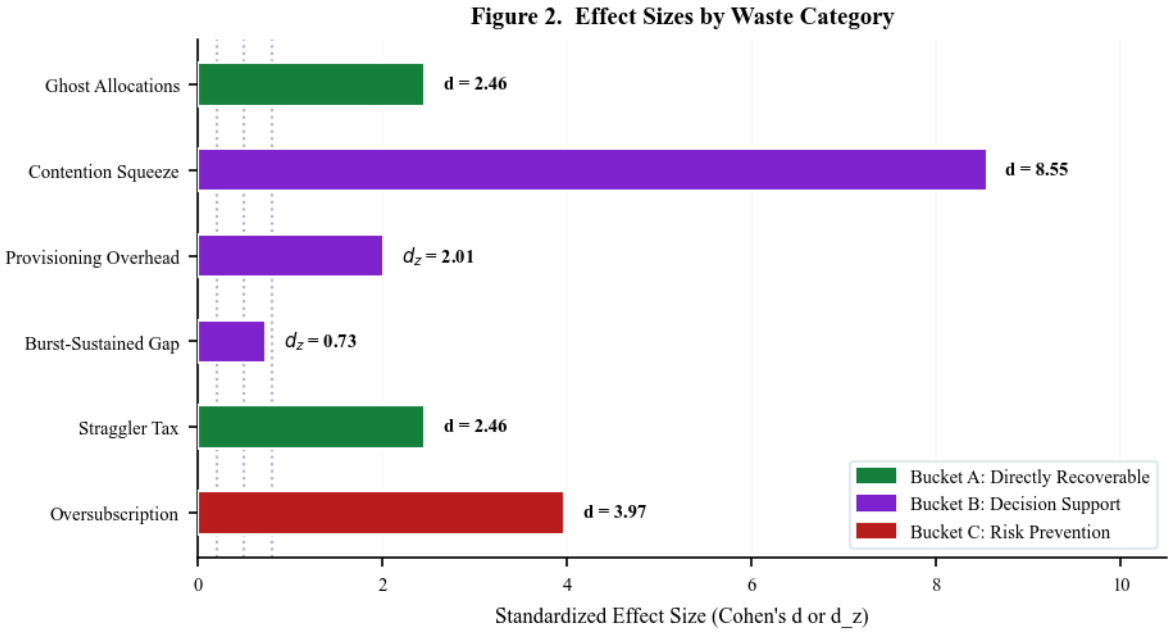

Figure 1: Figure 2. Standardized effect sizes across categories, colored by operational action bucket.

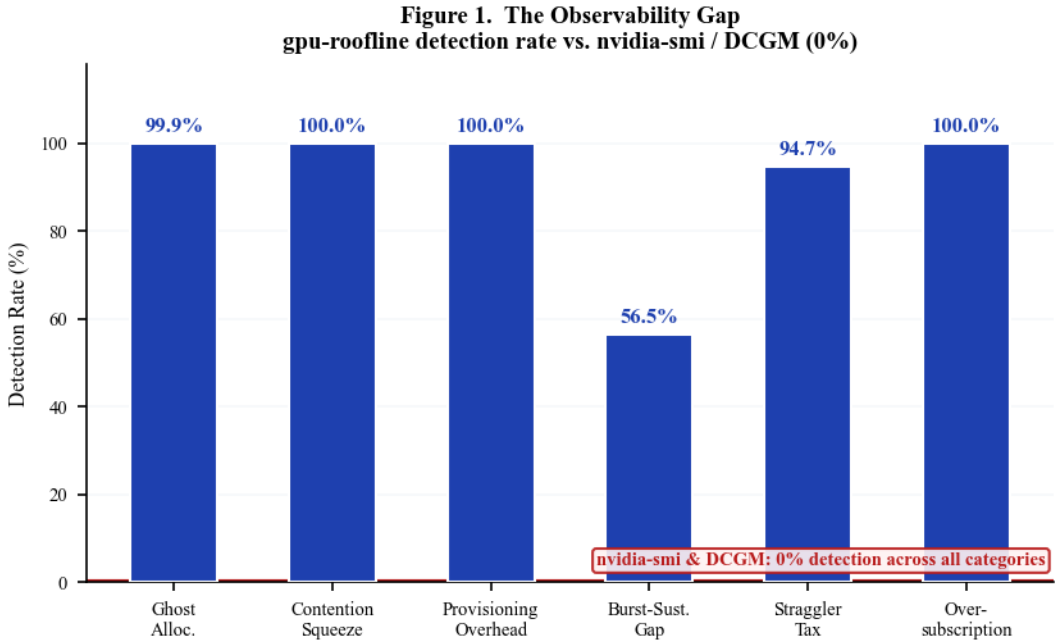

Figure 2: Figure 1. Detection rates by monitoring tool across all six waste categories.

track per-tenant bandwidth, time provisioning transitions, measure thermal trajectories, correlate fleet-wide barriers, or sum allocations against physical limits. These capabilities cannot be enabled through configuration. They require fundamentally different measurement architectures.

The `gpu-roofline` detection rate varies by category. The weakest performance occurs in the burst-to-sustained gap (56.5%), where small thermal gaps below the 1% detection threshold are missed. The strongest performance occurs in contention, provisioning, and oversubscription (100%), where the signal-to-noise ratio is high.

1.6.4 5.4 Operational Impact: Three Action Buckets

The six waste categories do not all represent the same type of problem and they do not all have the same solution. We categorize them into three action buckets based on what visibility enables (Figure 3).

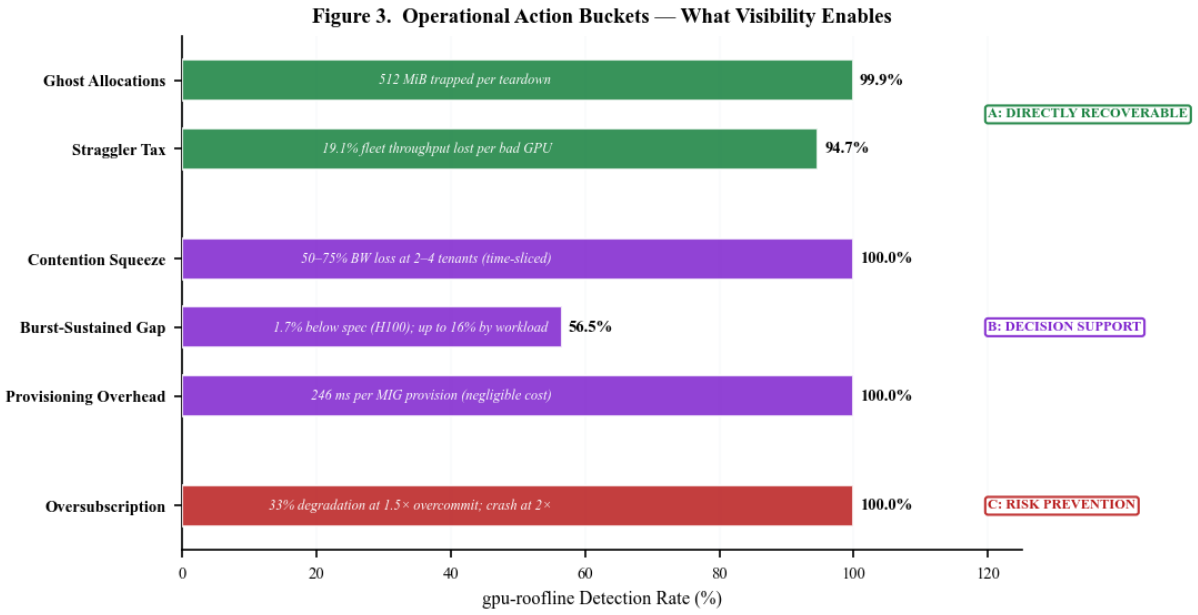

Figure 3: Figure 3. Three operational action buckets with detection rates and per-event impact.

1.6.4.1 Bucket A: Directly Recoverable Capacity Ghost allocations trap VRAM that could host additional tenant instances. The simulation yields a median of 512 MiB trapped per teardown event. At 20 teardowns per day, this represents approximately 10 GiB of VRAM per GPU per day that can be reclaimed once the ghost is detected. Detection rate: 99.9%.

The straggler tax wastes fleet-wide GPU-hours at synchronization barriers. A single degraded GPU in an 8-GPU training job wastes 16.7% of fleet throughput as the other 7 GPUs idle at the barrier. This effect scales multiplicatively with fleet size. At 128 GPUs, one straggler wastes the equivalent of 24 GPUs at every sync barrier (Figure 4). Identifying the straggler (detection rate: 94.7%) and replacing or reassigning it immediately recovers this capacity.

These two categories represent genuine capacity recovery. Detect the problem, take action, free the resource.

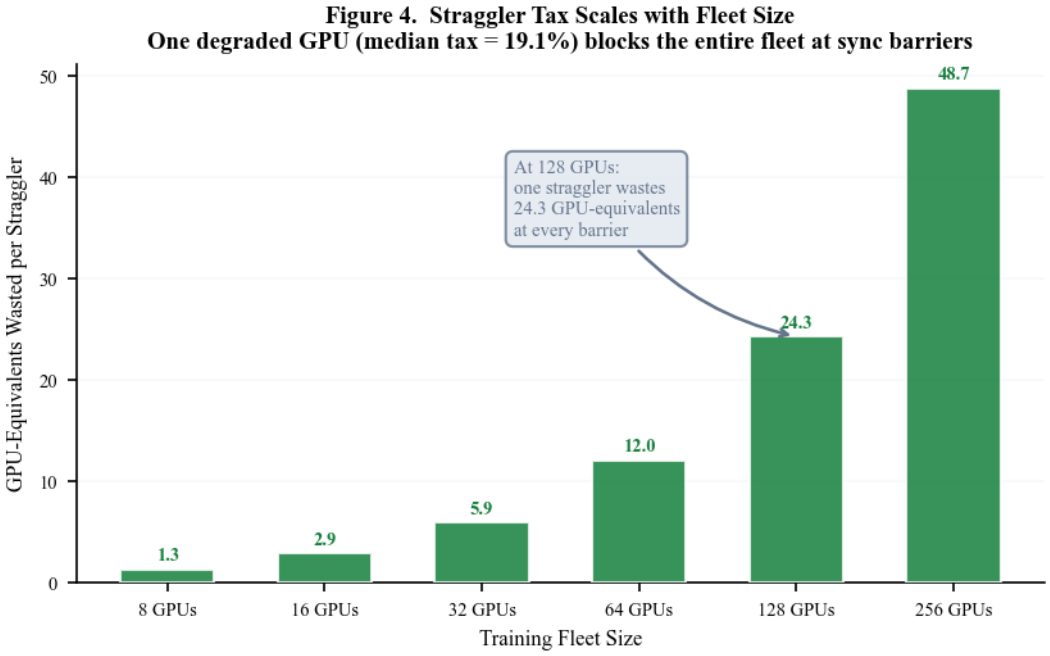

Figure 4: Figure 4. Straggler tax scales multiplicatively with fleet size.

1.6.4.2 Bucket B: Decision Support Contention squeeze is the inherent cost of time-slicing. With 2 tenants, each receives approximately 50% of available bandwidth. With 4 tenants, each receives approximately 25% (Figure 5). This degradation cannot be eliminated. It is the physics of shared access. Measuring it enables operators to make informed infrastructure decisions: deploy MIG with hardware isolation and guaranteed bandwidth for latency-sensitive workloads, and use time-slicing with higher density for throughput-tolerant batch jobs. Without per-tenant bandwidth visibility, this decision is made without data.

The burst-to-sustained gap is thermal physics. H100 SXM GPUs exhibit a median 1.7% gap between burst and sustained performance. Compute-bound workloads on some profiles exhibit gaps up to 16%. This cannot be fixed, but it can be priced correctly. Cloud providers advertising peak specifications for sustained workloads are implicitly overcommitting on performance. Measuring the actual sustained ceiling enables accurate SLAs.

Provisioning overhead (246 ms per MIG provision) is measurable but economically negligible at less than $1 per GPU per year.

1.6.4.3 Bucket C: Risk Prevention Oversubscription occurs when aggregate vGPU VRAM allocations exceed physical GPU capacity. At 1.5x overcommit, tenants experience 33% performance degradation. At 2x overcommit, out-of-memory crashes become likely. Each tenant allocation appears successful individually. No tool sums them against the physical limit. Detection enables enforcement before tenants are silently impacted.

1.6.5 5.5 Hypothesis Summary

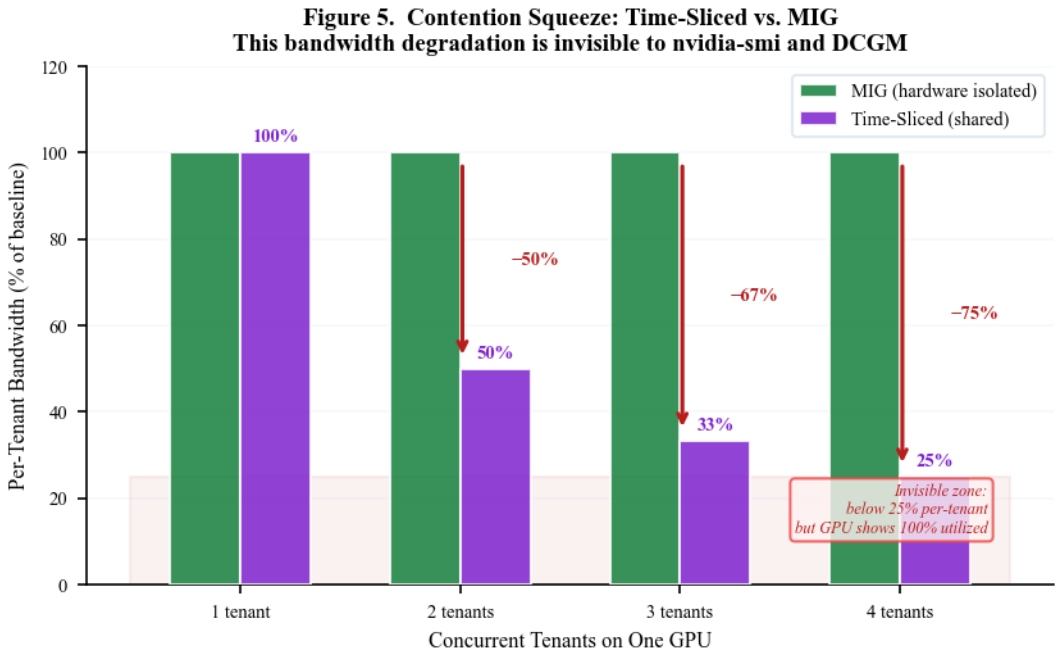

Figure 5: Figure 5. Per-tenant bandwidth degradation under time-slicing versus MIG isolation.

Hypothesis	Prediction	Result
H1: All six categories produce significant waste	Significant effects in all 6	**Supported.** All d/d_z > 0.73, all Holm p < 1e-300
H2: Existing tools miss >80% of waste	<20% detection by nvidia-smi/DCGM	**Supported.** 0% detection across all categories
H3: Economic impact > $0 per GPU per year	Bootstrap CI lower bound > $0	**Supported.** Lower bound $8,149/GPU/yr (8-GPU fleet)

1.7 6. Discussion

1.7.1 6.1 A Monitoring Paradigm Shift

The GPU monitoring paradigm emerged in the single-device, single-tenant era. `nvidia-smi` answered the questions that mattered when one workload ran on one GPU: Is this device functional? Is it overheating? Has it encountered an ECC error? The industry has moved to multi-tenant virtualization and fleet-scale distributed training, but the monitoring architecture has not followed. This study quantifies the consequences of that gap.

The six waste categories identified in this study are not six independent problems. They are one structural problem observed at three scales. At the device scale, thermal physics creates a gap between advertised and deliverable performance. At the virtualization scale, lifecycle transitions

and bandwidth sharing create losses that per-device metrics cannot capture. At the fleet scale, synchronization dependencies amplify single-device degradation into cluster-wide waste. The common thread is that all six require measurement across time, across tenants, or across devices. Current tools provide none of these measurement capabilities.

The analogy is straightforward. Operating a GPU fleet with `nvidia-smi` alone is comparable to operating a power grid with meters that show voltage but not consumption. The equipment appears healthy while efficiency losses accumulate unmeasured.

1.7.2 6.2 Operational Priorities

The three-bucket model translates directly into deployment priority for operators building monitoring capability.

Straggler detection offers the highest return. A single degraded GPU in a 128-GPU training cluster wastes 127 GPU-equivalents of blocked time at every synchronization barrier. The return on detecting and replacing one straggler is immediate and scales multiplicatively with fleet size. For organizations running large-scale distributed training, this category alone justifies the investment in fleet-wide performance monitoring.

Oversubscription detection addresses the highest risk. Silent overcommit is the only category that can cause direct customer-visible failures through out-of-memory crashes. The probability may be low in well-managed environments. The reputational cost of unexplained tenant failures in a cloud offering is disproportionately high.

Contention visibility provides the highest strategic value. The decision between MIG and time-slicing determines tenant density, performance guarantees, and pricing models. Without per-tenant bandwidth data, this decision is driven by vendor guidance rather than measured workload characteristics. Operators who can measure per-tenant bandwidth can optimize partitioning strategies for their specific workload mix.

1.7.3 6.3 Limitations and Validity

The noise model parameters are based on published NVML instrumentation characteristics, not empirical calibration against live systems. The thermal model uses Newton's law of cooling with published TDP specifications. Real hardware may exhibit behaviors not captured by these models.

The 0% detection rate for `nvidia-smi` and DCGM is a modeling assumption based on documented capabilities. Live evaluation may reveal edge cases of partial detection through DCGM custom metric plugins.

With 10,000 to 20,000 observations per group, statistical significance is guaranteed for any non-zero effect. We emphasize effect sizes and confidence intervals over p-values throughout. Readers should evaluate practical significance against their specific operational context.

Results are specific to the simulated GPU profiles (primarily H100 SXM, with H200 and consumer GPUs in the burst-sustained category) and three workload types (compute-bound, memory-bound, mixed). Real-world workloads are more diverse. Fleet utilization patterns vary across providers. The cost model parameters should be treated as substitutable inputs rather than universal constants.

1.8 7. Reproducibility

Full reproducibility is a requirement of this work, not a convenience feature. The simulation is fully deterministic. Re-execution with seed 42 produces byte-identical output:

`SHA-256: e386a3d599a238901abff636d421dbcb095cca31616e425d05e5653744fd3912`

We verified this by independent re-execution during the analysis phase.

1.8.1 7.1 Artifact Inventory

Artifact	Description
`simulation-raw.json` (83.7 MB)	120,000 trial-level records with IVs, DVs, and detection outcomes
`analyze_study.py` (1,280 lines)	Full analysis pipeline: statistics, cost model, provenance
`summary.md`	Main results tables (Tables 1 through 4)
`supplement.md`	Supplementary tables (S1 through S5)
`PROVENANCE.md`	Software versions, commands, SHA-256 chain
`SHA256SUMS.txt`	Per-file checksums for all artifacts

1.8.2 7.2 Rerun Commands

```
# Simulation (0.4s, produces identical output)
cargo run -p gpu-harness --release --bin study_sim -- \
  --out docs/study-results/simulation-raw.json

# Analysis (requires Python 3.11+, numpy, pandas, scipy)
python scripts/analyze_study.py \
  --input docs/study-results/simulation-raw.json \
  --output-root study-output --repo-root . \
  --bootstrap-resamples 10000 --seed 42
```

1.9 8. Conclusion

GPU efficiency loss spans every layer of the operations stack, and so does the observability gap. We demonstrate that six categories of measurable waste are completely invisible to the standard monitoring tools deployed across the industry. `nvidia-smi` and DCGM detect 0% of waste events across all six categories. Purpose-built instrumentation detects 56.5 to 100%.

We provide strong experimental evidence that these losses are quantifiable and large (Cohen's d/d_z from 0.73 to 8.55). They span device-level thermal physics, virtualization lifecycle management, and fleet-level distributed training coordination.

The efficiency gap requires three distinct operational responses. Ghost allocations and straggler GPUs represent directly recoverable capacity. Contention and thermal throttling are governed by physics that cannot be eliminated but can be measured, enabling better infrastructure decisions. Oversubscription is a preventable risk.

To address this gap, comprehensive hardware validation studies must be conducted across datacenter-class systems under production conditions. We publish the complete protocol, analysis pipeline, and quantitative predictions as an open invitation to GPU cloud operators, system integrators, and NVIDIA.

1.10 References

[1] NVIDIA, "Multi-Instance GPU User Guide," NVIDIA Documentation, 2024.

[2] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65-76, 2009.

[3] A. Harlap et al., "Addressing the straggler problem for iterative convergent parallel ML," *Proceedings of the Seventh ACM Symposium on Cloud Computing (SoCC)*, 2016.

[4] NVIDIA, "NVIDIA Data Center GPU Manager (DCGM) Documentation," NVIDIA Documentation, 2024.

[5] NVIDIA, "NVIDIA System Management Interface (nvidia-smi) Documentation," NVIDIA Documentation, 2024.

2 The Hidden Cost of GPU Virtualization

2.1 Simulation Dataset

- Input: `D:/study-output-v4/raw/simulation-raw.json`
- Phase: `simulation`
- Protocol version: `1.0`
- Seed: `42`
- Total trials: `120000`
- Target trials/category: `20000`
- Claim framing: simulation evidence, modeled tool visibility, and scenario-based economic impact. Hardware validation remains future work.

2.2 Table 1. What Visibility Enables — Three Action Buckets

Each waste category maps to a different type of operational action. These are NOT additive — they apply to different fractions of GPU operating time.

Bucket A — Directly Recoverable: Capacity that can be freed or hardware that can be replaced once the problem is detected.

Bucket B — Decision Support: Physics you cannot change, but knowing the numbers lets you choose MIG vs time-slicing, price per-tenant correctly, and plan capacity.

Bucket C — Risk Prevention: Silent degradation you can detect and stop before it impacts tenants.

Bucket	Category	What Visibility Gives You	Per-Event Magnitude	Per-GPU Impact (example)	Fleet Multiplier
A: Directly Recoverable	Ghost allocations	Free trapped VRAM for new tenants	512 MiB trapped/teardown	10224 MiB freed/day @ 20 teardowns	Linear with teardown frequency
A: Directly Recoverable	Straggler tax	Identify and replace degraded GPUs	19.1% fleet throughput lost per straggler	16.7% fleet capacity wasted in 8-GPU job	Multiplies with fleet size (N-1 GPUs idle at barrier)
B: Decision Support	Contention squeeze	Choose MIG vs time-slicing; price per-tenant correctly	66.7% bandwidth loss at median tenant count	~50% loss at 2 tenants, ~67% at 3, ~75% at 4 (time-sliced)	Per-GPU; scales with multi-tenancy ratio
B: Decision Support	Burst-sustained gap	Set honest SLAs; avoid overcommit-ting on specs	1.7% below advertised peak (H100 median)	1-16% depending on workload type and GPU model	Per-GPU; higher for consumer GPUs with weaker cooling
B: Decision Support	Provisioning overhead	Measure dead time during MIG partition creation	246 ms per provision event	Economically negligible (<$1/GPU/yr)	Linear with provision frequency
C: Risk Prevention	Oversubscription	Detect before tenants experience silent degradation	33.3% degradation when overcom-mitted	33% perf loss at 1.5x overcommit; crash risk at 2x	Per-GPU; risk compounds with instance count

2.3 Table 2. Detection Rates by Tool Across All 6 Waste Categories

The core finding: `nvidia-smi` and DCGM provide **zero visibility** into all six waste categories. `gpu-roofline` detects 56.5–100% of waste events across categories.

Category	nvidia-smi Detection Rate	DCGM Detection Rate	gpu-roofline Detection Rate	McNemar p (smi vs roofline)	McNemar p (dcgm vs roofline)
Ghost allocations	0.0	0.0	99.9	<1e-300	<1e-300
Contention squeeze	0.0	0.0	100.0	<1e-300	<1e-300
Provisioning overhead	0.0	0.0	100.0	<1e-300	<1e-300

Category	nvidia-smi Detection Rate	DCGM Detection Rate	gpu-roofline Detection Rate	McNemar p (smi vs roofline)	McNemar p (dcgm vs roofline)
Burst-to-sustained gap	0.0	0.0	56.5	<1e-300	<1e-300
Straggler tax	0.0	0.0	94.7	<1e-300	<1e-300
Oversubscription	0.0	0.0	100.0	<1e-300	<1e-300
Overall	0.0	0.0	88.4	<1e-300	<1e-300

2.4 Table 3. Cross-Category Omnibus Statistical Tests

Category	N (a)	N (b)	Design	Primary Test	Metric	Median (a)	Median (b)	Statistic	Raw p-value	Holm p-value	Cohen's d / d_z	Rank-biserial r	95% CI
Ghost allocations	10000	10000	independent	Mann-Whitney U	ghost_bytes_mean	536350.00	826500.00	99370861	<1e-300	<1e-300	2.46	1.00	[527280091.27, 545228612.40]
Contention squeeze	10000	10000	independent	Mann-Whitney U	bandwidth_loss_pct	66.66	0.00	100000000	<1e-300	<1e-300	8.55	1.00	[66.61, 66.68]
Provisioning overhead	20000	20000	paired	Wilcoxon signed-rank	spin_up_latency_ms	246.59	0.50	200010000	<1e-300	<1e-300	2.01	1.00	[244.32, 248.26]
Burst-to-sustained gap	20000	20000	paired	Wilcoxon signed-rank	gap_pct	1.69	0.00	87021028	<1e-300	<1e-300	0.73	1.00	[1.63, 1.77]
Straggler tax	10000	10000	independent	Mann-Whitney U	straggler_tax_pct	19.09	4.28	99805998	<1e-300	<1e-300	2.46	1.00	[14.56, 15.09]
Oversubscription	10000	10000	independent	Mann-Whitney U	performance_degradation_pct	33.33	0.00	100000000	<1e-300	<1e-300	3.97	1.00	[33.33, 33.33]

2.5 Table 4. Scenario-Based Annual Impact by Category and Fleet Scale

These figures are scenario outputs under explicit assumptions ($2.50/GPU/hr, specific fleet utilization parameters). They are NOT directly additive across categories. See the sensitivity analysis in `derived/supplement.md`.

Waste Category	Effect Size (d/d_z)	Waste per Event	Events/Day/GPU	Annual $/GPU	8-GPU Fleet	100-GPU Fleet	10,000-GPU Fleet
Ghost allocations	2.46	511.5 MiB	20.00	$2,733	$5,466	$273,321	$34,165,064
Contention squeeze	8.55	66.66%	0.83	$9,124	$38,929	$912,394	$114,700,996
Provisioning overhead	2.01	246.09 ms	20.00	$1	$2	$125	$31,190
Burst-to-sustained gap	0.73	1.69%	0.75	$278	$1,481	$27,762	$3,084,702
Straggler tax	2.46	14.81%	2.00	$5,518	$9,755	$551,828	$139,336,568
Oversubscription	3.07	33.33%	0.50	$3,650	$9,733	$365,000	$48,666,667
Total				$21,304	$65,366	$2,130,430	$339,985,187

2.6 Cost Model Bootstrap Confidence Intervals

Scale	Point estimate / GPU / year	95% CI low / GPU / year	95% CI high / GPU / year
8-GPU Fleet	$8,171	$8,149	$8,192
100-GPU Fleet	$21,304	$21,215	$21,389
10,000-GPU Fleet	$33,999	$33,809	$34,193

2.7 Interpretation Guardrails

- Bucket A categories (ghost, straggler) represent directly actionable capacity recovery.
- Bucket B categories (contention, burst gap) are physics — they cannot be eliminated, but measuring them enables better pricing, SLA, and partitioning decisions.
- Bucket C (oversubscription) is risk prevention — detect before tenants experience silent degradation.
- Scenario dollar figures in Table 4 assume freed or avoided capacity can be monetized at $2.50/GPU/hr.
- Category-specific tests and sensitivity analyses are in `derived/supplement.md`.

3 Supplementary Tables

3.1 Table S1. Descriptive Statistics by Category and Condition

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Ghost allocations	control	control_1gb	3304	391029.23	-2784.00	4539758.28	998313.75	-2671489.00	66430929.00

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Ghost allocations	control	control_2g.30gb	3303	336134.65	13848.00	3799625.00	976113.00	-2502325.00	65815200.00
Ghost allocations	control	control_3g.40gb	3403	353060.30	15373.00	4043550.64	987936.00	-2703336.00	67814829.00
Ghost allocations	treatment	clean_1g.10gb	1012	530885879.76	3343192.00	1500791958.80	475226.00	-121201.00	1075175500.00
Ghost allocations	treatment	clean_2g.20gb	1011	542395594.39	3923658.00	1087100475.87	368033.00	-629284.00	1073507951.00
Ghost allocations	treatment	clean_3g.40gb	1011	532539547.56	5214024.00	1076833552.78	496122.85	-155154.00	1073762452.00
Ghost allocations	treatment	rapid_churn_1g.10gb	1111	529905112.52	7027181.00	1092722905.77	153933.00	-15948.00	1121729515.00
Ghost allocations	treatment	rapid_churn_2g.20gb	1112	548965538.29	8126770.00	1095658705.62	357483.50	-23798.00	1109296215.00
Ghost allocations	treatment	rapid_churn_3g.40gb	1113	539175321.57	9461259.00	1040661985.30	414972.50	-4790.00	1073566349.00
Ghost allocations	treatment	under_load_1g.10gb	1111	523338447.52	7179090.00	1061474005.28	214374.45	-39539.00	1068458951.00
Ghost allocations	treatment	under_load_2g.20gb	1112	549621948.52	8468193.00	1090142865.94	462875.00	-23951.00	1073611128.00
Ghost allocations	treatment	under_load_3g.40gb	1113	538731790.47	3319153.00	1045566615.26	861874.00	-229079.00	1099223345.00
Contention squeeze	control	mig_1_tenants	1250	0.77	0.00	1.10	1.32	0.00	6.03
Contention squeeze	control	mig_2_tenants	1250	0.58	0.00	0.85	0.93	0.00	5.31
Contention squeeze	control	mig_3_tenants	1250	0.50	0.05	0.70	0.85	0.00	3.78
Contention squeeze	control	mig_4_tenants	1250	0.41	0.00	0.62	0.66	0.00	3.47
Contention squeeze	control	timesliced_100_tenants	500	0.79	0.00	1.16	1.30	0.00	6.94
Contention squeeze	treatment	timesliced_234_tenants	32	50.00	50.00	0.72	0.96	47.48	52.41

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Contention squeeze	treatment	timesliced_3_tenants	333	66.66	66.66	0.39	0.53	65.31	67.90
Contention squeeze	treatment	timesliced_3_tenants	333	75.01	75.01	0.25	0.33	74.10	75.95
Provisioning overhead	paired	1g.10gb_cold_0	667	125.39	119.71	39.27	47.95	49.33	387.53
Provisioning overhead	paired	1g.10gb_cold_1	667	142.10	136.66	44.39	56.10	54.79	388.21
Provisioning overhead	paired	1g.10gb_cold_3	667	173.30	167.05	54.81	65.53	63.65	435.23
Provisioning overhead	paired	1g.10gb_cold_6	667	217.83	211.36	65.81	78.11	82.58	590.08
Provisioning overhead	paired	1g.10gb_hot_0	667	210.84	201.25	62.91	68.68	86.66	499.36
Provisioning overhead	paired	1g.10gb_hot_1	667	225.23	214.82	69.63	85.00	69.38	689.03
Provisioning overhead	paired	1g.10gb_hot_3	667	255.62	245.11	80.79	95.40	98.83	696.86
Provisioning overhead	paired	1g.10gb_hot_6	667	306.46	290.04	95.03	118.62	123.71	810.78
Provisioning overhead	paired	1g.10gb_warm_0	667	154.80	145.48	49.44	63.03	57.55	472.70
Provisioning overhead	paired	1g.10gb_warm_1	667	173.81	165.09	53.94	71.25	68.76	445.74
Provisioning overhead	paired	1g.10gb_warm_3	667	203.99	197.67	61.98	82.85	66.41	512.75
Provisioning overhead	paired	1g.10gb_warm_6	667	253.75	247.62	74.95	100.11	89.21	678.88
Provisioning overhead	paired	2g.20gb_cold_0	667	189.33	180.12	57.06	70.90	64.98	401.49
Provisioning overhead	paired	2g.20gb_cold_1	667	200.57	191.58	62.94	81.95	80.24	567.24

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Provisioning overhead	paired	2g.20gb_hot	667	264.79	253.64	78.29	98.11	84.95	635.84
Provisioning overhead	paired	2g.20gb_hot	667	282.71	268.51	85.79	118.49	118.66	675.51
Provisioning overhead	paired	2g.20gb_warm	667	219.84	207.36	69.02	91.20	89.72	513.09
Provisioning overhead	paired	2g.20gb_warm	667	236.99	226.95	74.09	89.01	90.98	603.25
Provisioning overhead	paired	3g.40gb_cold	667	265.43	254.64	79.39	103.43	101.10	544.64
Provisioning overhead	paired	3g.40gb_cold	667	273.55	258.06	81.36	107.54	112.20	589.33
Provisioning overhead	paired	3g.40gb_hot	666	345.75	327.17	107.42	126.52	139.57	889.95
Provisioning overhead	paired	3g.40gb_hot	666	358.51	339.94	109.53	141.09	133.93	921.17
Provisioning overhead	paired	3g.40gb_warm	666	289.94	281.52	82.55	108.22	120.54	634.63
Provisioning overhead	paired	3g.40gb_warm	666	314.90	300.52	96.23	122.66	139.60	818.24
Provisioning overhead	paired	4g.40gb_cold	666	339.79	327.04	104.93	123.41	110.23	905.43
Provisioning overhead	paired	4g.40gb_hot	666	418.28	393.17	132.24	160.66	160.08	905.93
Provisioning overhead	paired	4g.40gb_warm	666	357.74	338.03	112.48	149.18	145.09	817.77
Provisioning overhead	paired	7g.80gb_cold	666	473.96	456.11	147.51	173.33	164.70	1232.44
Provisioning overhead	paired	7g.80gb_hot	666	548.15	523.58	159.79	210.58	209.86	1249.23

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Provisioning overhead	paired	7g.80gb_warm_0	666	501.29	475.38	159.80	193.57	183.96	1243.03
Burst-to-sustained gap	paired	h100_sxm_1334_compute_bound	1334	15.96	15.97	2.32	3.11	8.48	24.99
Burst-to-sustained gap	paired	h100_sxm_1334_memory_bound	1334	1.05	0.00	1.59	1.72	0.00	7.97
Burst-to-sustained gap	paired	h100_sxm_1334_mixed	1334	1.12	0.00	1.61	1.92	0.00	10.01
Burst-to-sustained gap	paired	h200_sxm_1334_compute_bound	1334	15.98	15.98	2.32	3.12	8.34	23.10
Burst-to-sustained gap	paired	h200_sxm_1334_memory_bound	1334	1.10	0.00	1.59	1.91	0.00	8.67
Burst-to-sustained gap	paired	h200_sxm_1333_mixed	1333	1.07	0.00	1.59	1.73	0.00	9.28
Burst-to-sustained gap	paired	mi300x_compute_bound	1333	20.98	21.04	2.23	2.96	14.63	28.11
Burst-to-sustained gap	paired	mi300x_memory_bound	1333	1.06	0.00	1.60	1.77	0.00	8.39
Burst-to-sustained gap	paired	mi300x_mixed	1333	1.04	0.06	1.52	1.74	0.00	8.12
Burst-to-sustained gap	paired	rtx_4090_1333_compute_bound	1333	4.52	4.50	2.54	3.65	0.00	14.12

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Burst-to-sustained gap	paired	rtx_4090_1333_memory_bound	333	1.08	0.00	1.58	1.83	0.00	9.07
Burst-to-sustained gap	paired	rtx_4090_1333_mixed	333	1.11	0.00	1.62	1.88	0.00	8.05
Burst-to-sustained gap	paired	rtx_5090_1333_compute_bound	333	6.54	6.58	2.61	3.53	0.00	15.38
Burst-to-sustained gap	paired	rtx_5090_1333_memory_bound	333	1.12	0.00	1.66	1.96	0.00	9.68
Burst-to-sustained gap	paired	rtx_5090_1333_mixed	333	1.24	0.27	1.72	2.14	0.00	8.83
Straggler tax	control	control_fleet128	333	5.18	5.09	0.84	1.09	2.99	10.02
Straggler tax	control	control_fleet32	333	4.22	4.12	1.03	1.35	1.34	8.94
Straggler tax	control	control_fleet64	333	3.16	3.05	1.26	1.72	0.26	8.88
Straggler tax	treatment	clock_stuck_sev0_fleet128	222	18.10	18.05	1.63	2.11	13.83	22.49
Straggler tax	treatment	clock_stuck_sev0_fleet32	222	18.32	18.05	1.81	2.40	12.39	23.54
Straggler tax	treatment	clock_stuck_sev0_fleet64	222	18.31	18.22	1.82	2.77	12.59	22.44
Straggler tax	treatment	clock_stuck_sev1_fleet128	222	34.27	34.21	1.27	1.86	30.44	37.33
Straggler tax	treatment	clock_stuck_sev1_fleet32	222	34.42	34.47	1.35	1.73	30.96	37.39
Straggler tax	treatment	clock_stuck_sev1_fleet64	222	34.39	34.36	1.47	1.84	29.87	39.88
Straggler tax	treatment	clock_stuck_sev2_fleet128	222	40.21	40.13	1.19	1.62	37.44	43.36
Straggler tax	treatment	clock_stuck_sev2_fleet32	222	40.32	40.15	1.29	1.84	36.75	43.28
Straggler tax	treatment	clock_stuck_sev2_fleet64	222	40.30	40.13	1.39	1.99	35.65	42.84
Straggler tax	treatment	memory_subsystem_sev0_fleet128	222	15.09	14.93	1.60	2.17	10.91	19.15

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Straggler tax	treatment	memory_subsystem_sev0_fleet128	222	15.05	14.96	1.90	2.29	8.11	20.98
Straggler tax	treatment	memory_subsystem_sev0_fleet8	222	14.93	14.98	1.85	2.26	9.97	19.78
Straggler tax	treatment	memory_subsystem_sev1_fleet128	222	29.93	30.86	1.29	2.02	26.62	32.96
Straggler tax	treatment	memory_subsystem_sev1_fleet8	222	29.97	29.86	1.30	1.64	25.55	34.44
Straggler tax	treatment	memory_subsystem_sev2_fleet128	222	30.12	30.23	1.58	2.18	25.79	33.76
Straggler tax	treatment	memory_subsystem_sev2_fleet8	222	49.91	49.88	1.00	1.37	47.15	53.41
Straggler tax	treatment	memory_subsystem_sev3_fleet128	222	50.02	50.04	1.08	1.51	47.49	53.26
Straggler tax	treatment	memory_subsystem_sev3_fleet8	222	50.10	50.11	1.04	1.29	47.61	53.02
Straggler tax	treatment	nvlink_sev0_fleet128	222	8.34	8.21	1.75	2.64	4.57	13.23
Straggler tax	treatment	nvlink_sev0_fleet32	222	8.40	8.40	1.80	2.38	3.60	12.63
Straggler tax	treatment	nvlink_sev0_fleet8	223	8.28	8.14	2.00	2.72	3.44	14.73
Straggler tax	treatment	nvlink_sev1_fleet128	222	16.81	16.73	1.83	2.77	12.99	21.64
Straggler tax	treatment	nvlink_sev1_fleet32	222	16.60	16.59	1.58	2.01	10.76	21.38
Straggler tax	treatment	nvlink_sev1_fleet8	222	16.78	16.77	1.83	2.32	11.72	22.24
Straggler tax	treatment	nvlink_sev2_fleet128	222	33.29	33.36	1.35	1.77	30.05	36.86
Straggler tax	treatment	nvlink_sev2_fleet32	222	33.33	33.28	1.44	1.82	28.81	38.34
Straggler tax	treatment	nvlink_sev2_fleet8	222	33.35	33.24	1.61	2.26	29.10	38.02
Straggler tax	treatment	pcie_fallback_sev0_fleet128	222	15.08	15.20	1.55	2.12	10.69	18.76
Straggler tax	treatment	pcie_fallback_sev0_fleet32	222	14.98	14.95	1.63	1.95	11.05	20.44
Straggler tax	treatment	pcie_fallback_sev0_fleet8	222	15.22	15.08	1.92	2.75	11.09	19.88
Straggler tax	treatment	pcie_fallback_sev1_fleet128	222	22.50	22.51	1.65	2.46	17.92	27.91
Straggler tax	treatment	pcie_fallback_sev1_fleet32	222	22.52	22.54	1.43	1.91	18.33	25.91
Straggler tax	treatment	pcie_fallback_sev1_fleet8	222	22.12	22.04	1.80	2.38	17.19	26.97

Category	Arm	Condition	N	Mean	Median	SD	IQR	Min	Max
Straggler tax	treatment	pcie_fallback_sev2_fleet128	222	26.35	26.38	1.46	2.19	22.70	30.45
Straggler tax	treatment	pcie_fallback_sev2_fleet32	222	26.41	26.46	1.47	1.85	22.04	30.28
Straggler tax	treatment	pcie_fallback_sev2_fleet8	222	26.13	26.05	1.62	2.16	21.04	31.12
Straggler tax	treatment	thermal_throttle_sev0_fleet128	223	16.07	16.02	1.53	2.06	9.72	20.43
Straggler tax	treatment	thermal_throttle_sev0_fleet32	223	15.93	15.86	1.70	2.47	12.11	20.94
Straggler tax	treatment	thermal_throttle_sev0_fleet8	223	15.87	15.97	1.98	2.41	9.36	21.59
Straggler tax	treatment	thermal_throttle_sev1_fleet128	223	16.25	16.27	1.75	2.50	10.74	20.48
Straggler tax	treatment	thermal_throttle_sev1_fleet32	223	16.07	16.14	1.74	2.27	11.13	21.10
Straggler tax	treatment	thermal_throttle_sev1_fleet8	223	15.99	15.96	1.89	2.61	11.60	21.50
Straggler tax	treatment	thermal_throttle_sev2_fleet128	223	16.35	16.39	1.50	1.93	13.01	19.98
Straggler tax	treatment	thermal_throttle_sev2_fleet32	223	16.15	16.19	1.85	2.10	11.25	22.82
Straggler tax	treatment	thermal_throttle_sev2_fleet8	223	15.93	15.85	1.88	2.72	11.39	20.74
Oversubscription	control	ratio1.00_instances1	2500	0.00	0.00	0.00	0.00	0.00	0.00
Oversubscription	control	ratio1.00_instances2	2500	0.00	0.00	0.00	0.00	0.00	0.00
Oversubscription	control	ratio1.00_instances4	2500	0.00	0.00	0.00	0.00	0.00	0.00
Oversubscription	control	ratio1.00_instances8	2500	0.00	0.00	0.00	0.00	0.00	0.00
Oversubscription	treatment	ratio1.25_instances1	834	20.00	20.00	0.00	0.00	20.00	20.00
Oversubscription	treatment	ratio1.25_instances2	834	20.00	20.00	0.00	0.00	20.00	20.00
Oversubscription	treatment	ratio1.25_instances4	834	20.00	20.00	0.00	0.00	20.00	20.00
Oversubscription	treatment	ratio1.25_instances8	834	20.00	20.00	0.00	0.00	20.00	20.00
Oversubscription	treatment	ratio1.50_instances1	833	33.33	33.33	0.00	0.00	33.33	33.33
Oversubscription	treatment	ratio1.50_instances2	833	33.33	33.33	0.00	0.00	33.33	33.33
Oversubscription	treatment	ratio1.50_instances4	833	33.33	33.33	0.00	0.00	33.33	33.33
Oversubscription	treatment	ratio1.50_instances8	833	33.33	33.33	0.00	0.00	33.33	33.33
Oversubscription	treatment	ratio2.00_instances1	833	50.00	50.00	0.00	0.00	50.00	50.00
Oversubscription	treatment	ratio2.00_instances2	833	50.00	50.00	0.00	0.00	50.00	50.00
Oversubscription	treatment	ratio2.00_instances4	833	50.00	50.00	0.00	0.00	50.00	50.00
Oversubscription	treatment	ratio2.00_instances8	833	50.00	50.00	0.00	0.00	50.00	50.00

3.2 Table S2. Full Statistical Test Results

Category	Comparison	Primary test	Statistic	Raw p	Adjusted p	Effect size	CI low	CI high	Point estimate
Ghost allocations	clean_1g.10gb vs control_1g.10gb	Mann-Whitney U	3704107.0000	<1e-300	<1e-300	0.9982	4976309865	5751692453	5334145976.0000
Ghost allocations	clean_2g.20gb vs control_2g.20gb	Mann-Whitney U	3698888.0000	<1e-300	<1e-300	0.9978	5075787195	5726314535	5502222733.0000
Ghost allocations	clean_3g.40gb vs control_3g.40gb	Mann-Whitney U	3700276.0000	<1e-300	<1e-300	0.9985	4912448315	5497740905	5219098651.0000
Ghost allocations	under_load_mg-1g.10gb vs control_1g.10gb	Mann-Whitney U	3702622.0000	<1e-300	<1e-300	0.9992	4838699205	5667829425	5275181874.0000
Ghost allocations	under_load_mg-2g.20gb vs control_2g.20gb	Mann-Whitney U	3700461.0000	<1e-300	<1e-300	0.9986	5270710565	5868219735	5582554345.0000
Ghost allocations	under_load_mg-3g.40gb vs control_3g.40gb	Mann-Whitney U	3701434.0000	<1e-300	<1e-300	0.9992	5149572765	5757357950	5401316542.0000
Ghost allocations	rapid_churn_mg-1g.10gb vs control_1g.10gb	Mann-Whitney U	3701130.0000	<1e-300	<1e-300	0.9984	4902116975	5658932105	5275029965.0000
Ghost allocations	rapid_churn_mg-2g.20gb vs control_2g.20gb	Mann-Whitney U	3701864.0000	<1e-300	<1e-300	0.9994	5262935025	5975492445	5585012922.0000
Ghost allocations	rapid_churn_mg-3g.40gb vs control_3g.40gb	Mann-Whitney U	3701583.0000	<1e-300	<1e-300	0.9993	5045665665	5625440805	5339597223.0000
Contention squeeze	1-4 time-sliced tenants	Friedman chi-square	9999.0000	<1e-300	<1e-300				
Contention squeeze	1 vs 2 tenants	Wilcoxon signed-rank	5556111.0000	<1e-300	<1e-300	1.0000			49.4627
Contention squeeze	2 vs 3 tenants	Wilcoxon signed-rank	5556111.0000	<1e-300	<1e-300	1.0000			16.6527
Contention squeeze	3 vs 4 tenants	Wilcoxon signed-rank	5556111.0000	<1e-300	<1e-300	1.0000			8.3531
Contention squeeze	timesliced vs MIG at 1 tenants	Wilcoxon signed-rank	226317.0000	0.1811	0.1811	0.0344			0.0000

Category	Comparison	Primary test	Statistic	Raw p	Adjusted p	Effect size	CI low	CI high	Point estimate
Contention squeeze	timesliced vs MIG at 2 tenants	Wilcoxon signed-rank	781875.0000	2.87e-206	2.87e-206	1.0000			49.5506
Contention squeeze	timesliced vs MIG at 3 tenants	Wilcoxon signed-rank	781875.0000	2.87e-206	2.87e-206	1.0000			66.3198
Contention squeeze	timesliced vs MIG at 4 tenants	Wilcoxon signed-rank	781875.0000	2.87e-206	2.87e-206	1.0000			74.7994
Provisioning overhead	Spin-up latency vs nvidia-smi reported latency	Paired t-test	284.8732	<1e-300	<1e-300	2.0144			246.0989
Provisioning overhead	Latency across load states	Kruskal-Wallis	1642.9355	<1e-300	<1e-300				
Provisioning overhead	Latency vs concurrent partitions	Linear regression	-12.0283	2.12e-115	2.12e-115	-0.1604			291.8689
Burst-to-sustained gap	Observed gap vs ideal 0% gap	Paired t-test	103.0821	<1e-300	<1e-300	0.7289			1.6902
Burst-to-sustained gap	Gap across workload types	Kruskal-Wallis	11997.9894	<1e-300	<1e-300				
Straggler tax	Treatment vs control	Mann-Whitney U	99805998.0000	<1e-300	<1e-300	0.9961			14.8060

Category	Comparison	Primary test	Statistic	Raw p	Adjusted p	Effect size	CI low	CI high	Point estimate
Straggler tax	Treatment across fleet sizes	Kruskal-Wallis	0.3605	0.8351	0.8351				
Straggler tax	Tax vs severity	Linear regression	9.3291	<1e-300	<1e-300	0.6778			14.5365
Oversubscription	Treatment vs control	Mann-Whitney U	100000000.000	<1e-300	<1e-300	1.0000			33.3333
Oversubscription	Degradation vs over-commit ratio	Linear regression	39.0496	<1e-300	<1e-300	0.9921			-27.3846
Oversubscription	Degradation across instance counts	Kruskal-Wallis	0.0000	1.0000	1.0000				

3.3 Table S4. Cost Model Parameter Sensitivity

Parameter	P10	P90	100-GPU Total @ P10	100-GPU Total @ P90	Range
ghost_rate	0.8995	1.0993	$2,103,098	$2,157,762	$54,664
avg_ghost_frac	0.0056	0.0069	$2,103,098	$2,157,762	$54,664
contention_drop_pct	59.9930	73.3248	$2,039,190	$2,221,669	$182,479
avg_spin_up_secs	0.2215	0.2707	$2,130,417	$2,130,442	$25
burst_sustained_gap_pct	1.5212	1.8593	$2,127,654	$2,133,206	$5,552
straggler_tax_pct	17.1802	20.9980	$2,075,247	$2,185,613	$110,366
oversub_waste_pct	30.0000	36.6667	$2,093,930	$2,166,930	$73,000

3.4 Table S5. Tool Detection Confusion Matrices

Category	Tool	TP	FP	TN	FN	Sensitivity %	Specificity %	Precision %	F1
Ghost allocations	gpu-roofline	9994	838	9162	6	99.94	91.62	92.26	0.96
Ghost allocations	nvidia-smi	0	0	10000	10000	0.00	100.00	0.00	0.00

Category	Tool	TP	FP	TN	FN	Sensitivity %	Specificity %	Precision %	F1
Ghost allocations	DCGM	0	0	10000	10000	0.00	100.00	0.00	0.00
Contention squeeze	gpu-roofline	10000	0	10000	0	100.00	100.00	100.00	1.00
Contention squeeze	nvidia-smi	0	0	10000	10000	0.00	100.00	0.00	0.00
Contention squeeze	DCGM	0	0	10000	10000	0.00	100.00	0.00	0.00
Provisioning overhead	gpu-roofline	20000	0	0	0	100.00	0.00	100.00	1.00
Provisioning overhead	nvidia-smi	0	0	0	20000	0.00	0.00	0.00	0.00
Provisioning overhead	DCGM	0	0	0	20000	0.00	0.00	0.00	0.00
Burst-to-sustained gap	gpu-roofline	11294	0	0	8706	56.47	0.00	100.00	0.72
Burst-to-sustained gap	nvidia-smi	0	0	0	20000	0.00	0.00	0.00	0.00
Burst-to-sustained gap	DCGM	0	0	0	20000	0.00	0.00	0.00	0.00
Straggler tax	gpu-roofline	9465	1	9999	535	94.65	99.99	99.99	0.97
Straggler tax	nvidia-smi	0	0	10000	10000	0.00	100.00	0.00	0.00
Straggler tax	DCGM	0	0	10000	10000	0.00	100.00	0.00	0.00
Oversubscription	gpu-roofline	10000	0	10000	0	100.00	100.00	100.00	1.00
Oversubscription	nvidia-smi	0	0	10000	10000	0.00	100.00	0.00	0.00
Oversubscription	DCGM	0	0	10000	10000	0.00	100.00	0.00	0.00
Overall	gpu-roofline	70753	839	39161	9247	88.44	97.90	98.83	0.93
Overall	nvidia-smi	0	0	40000	80000	0.00	100.00	0.00	0.00

Category	Tool	TP	FP	TN	FN	Sensitivity %	Specificity %	Precision %	F1
Overall	DCGM	0	0	40000	80000	0.00	100.00	0.00	0.00

3.5 Straggler Threshold Sweep

Threshold	Sensitivity %	Specificity %
3.03	100.00	20.00
3.91	99.96	39.96
4.59	99.88	59.88
5.29	99.73	79.73
6.97	98.38	98.38
14.99	80.00	100.00
17.26	60.00	100.00
24.38	40.00	100.00
33.84	20.00	100.00

- Table S3 remains reserved for hardware-phase calibration results.